

Error Debugging with AI — Systematic Approaches to Finding and Fixing Bugs

Name	S.Vishnu Teja
Enrollment No.	2503A52720
Batch	24
Course Code	25CAI102PC205

Task Description #1 (Syntax Errors – Missing Parentheses in Print Statement)

Task: Provide a Python snippet with a missing parenthesis in a print statement (e.g., `print "Hello"`). Use AI to detect and fix the syntax error.

Bug: Missing parentheses in print statement

```
def greet():  
    print "Hello, AI Debugging Lab!"  
greet()
```

Requirements:

- Run the given code to observe the error.
- Apply AI suggestions to correct the syntax.
- Use at least 3 assert test cases to confirm the corrected code works.

Expected Output #1:

Corrected code with proper syntax and AI explanation.

Code:

```
import io  
from contextlib import redirect_stdout
```

```
def greet():  
    print("Hello, AI Debugging Lab!")
```

Corrected program

```
greet()
```

```
def capture_output():  
    output = io.StringIO()  
    with redirect_stdout(output):  
        greet()  
    return output.getvalue()
```

Three assert test cases

```
assert capture_output() == "Hello, AI Debugging Lab!\n"  
assert capture_output().strip() == "Hello, AI Debugging Lab!"  
assert capture_output().startswith("Hello, AI Debugging Lab!")
```

Output:

Hello, AI Debugging Lab!

Task Description #2 (Incorrect condition in an If Statement)

Task: Supply a function where an if-condition mistakenly uses = instead of ==. Let AI identify and fix the issue.

Bug: Using assignment (=) instead of comparison (==)

```
def check_number(n):  
    if n = 10:  
        return "Ten"  
    else:  
        return "Not Ten"
```

Requirements:

- Ask AI to explain why this causes a bug.
- Correct the code and verify with 3 assert test cases.

Expected Output #2:

- Corrected code using == with explanation and successful test execution.

Code:

```
def check_number(n):
```

```
if n == 10:
    return "Ten"
else:
    return "Not Ten"
```

#test cases for the function

```
print(check_number(10)) # Expected output: "Ten"
print(check_number(5)) # Expected output: "Not Ten"
```

Output:

Ten

Not Ten

Task Description #3 (Runtime Error – File Not Found)

Task: Provide code that attempts to open a non-existent file and crashes. Use AI to apply safe error handling.

Bug: Program crashes if file is missing

```
def read_file(filename):
    with open(filename, 'r') as f:
        return f.read()
print(read_file("nonexistent.txt"))
```

Requirements:

- Implement a try-except block suggested by AI.
- Add a user-friendly error message.
- Test with at least 3 scenarios: file exists, file missing, invalid path.

Expected Output #3:

- Safe file handling with exception management.

Code:

Debugged file-reading program

```
def read_file(filename):
    try:
```

```

    with open(filename, "r", encoding="utf-8") as f:
        return f.read()
except FileNotFoundError:
    return f"Error: File '{filename}' was not found."
except (OSError, ValueError):
    return f"Error: The path '{filename}' is invalid or cannot be accessed."
except Exception as e:
    return f"Error: Something went wrong while reading '{filename}': {e}"

```

Tests

1) Existing file

```

with open("existing.txt", "w", encoding="utf-8") as f:
    f.write("Hello from existing.txt\n")
print(read_file("existing.txt"))

```

2) Missing file

```

print(read_file("nonexistent.txt"))

```

3) Invalid path

```

print(read_file("bad\0name.txt"))

```

Output:

Hello from existing.txt

Error: File 'nonexistent.txt' was not found.

Error: The path 'badname.txt' is invalid or cannot be accessed.

Task Description #4 (Calling a Non-Existent Method)

Task: Give a class where a non-existent method is called (e.g., `obj.undefined_method()`). Use AI to debug and fix.

Bug: Calling an undefined method

```
class Car:
    def start(self):
        return "Car started"

my_car = Car()
print(my_car.drive()) # drive() is not defined
```

Requirements:

- Students must analyze whether to define the missing method or correct the method call.
- Use 3 assert tests to confirm the corrected class works.

Expected Output #4:

- Corrected class with clear AI explanation.

Code:

```
class Car:
    def start(self):
        return "Car started"

    def drive(self):
        return "Car is driving"

my_car = Car()

# Three assert tests
assert my_car.start() == "Car started"
assert my_car.drive() == "Car is driving"
assert my_car.drive().startswith("Car")
print(my_car.start())
print(my_car.drive())
print("All tests passed for Car class.")
```

Output:

Car started

Car is driving

All tests passed for Car class.

Task Description #5 (TypeError – Mixing Strings and Integers in Addition)

Task: Provide code that adds an integer and string ("5" + 2) causing a TypeError. Use AI to resolve the bug.

Bug: TypeError due to mixing string and integer

```
def add_five(value):
```

```
    return value + 5
```

```
print(add_five("10"))
```

Requirements:

- Ask AI for two solutions: type casting and string concatenation.
- Validate with 3 assert test cases.

Expected Output #5:

- Corrected code that runs successfully for multiple inputs.

Note: Report should be submitted a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots

Code:

```
def add_five_numeric(value):
```

```
    return int(value) + 5
```

Solution 2: String concatenation

This keeps the value as a string and appends the integer converted to a string.

```
def add_five_string(value):
```

```
    return value + str(5)
```

Validate the corrected code with three assert tests

```
assert add_five_numeric("10") == 15
```

```
assert add_five_numeric("20") == 25
```

```
assert add_five_string("10") == "105"
```

```
print(add_five_numeric("10")) # Expected output: 15
```

```
print(add_five_string("10")) # Expected output: "105"
```

Output:

15

105